Shipment route selection under cost, time, and risk

Aiqon AI Internship | Technical report | 12 October 2026

Repository: https://github.com/vivekvar-dl/aiqon-route-optimizer

1. Decision framework

Routeplan selects from supplied candidate routes; it does not discover paths through a live transport network. Optimal means the lowest risk-adjusted landed cost among routes whose quoted schedule meets the shipment deadline. The shipment supplies its origin, destination, cargo value, deadline, priority, and routes with leg-level freight, time, delay probability, congestion, and geopolitical risk.

For scenario s, let F be freight, V cargo value, L(s) cargo loss, T(s) transit hours, and D the deadline. Holding price h and lateness price p are fractions of cargo value per day. Each scenario costs:

```
C(s) = F + L(s) + h * V * T(s) / 24
                + p * V * max(T(s) - D, 0) / 24
Score = mean(C) + a * [CVaR90(C) - mean(C)]
```

CVaR90 is the mean of the worst 10% of simulated costs. Tail weight a ranges from 0 to 1: zero optimizes the mean, while one optimizes that tail mean. The lowest score wins; ties break on freight plus expected loss, then route letter. All terms are dollars, so an explanation can reconcile a freight premium with savings in holding, lateness, and tail risk. A normalized weighted score would obscure those economic trade-offs and make rankings depend on the alternatives used for normalization.

Priority	Holding / day	Lateness / day	Tail weight
Cost-first	0.03% of V	0.2% of V	0.10
Balanced	0.10% of V	1% of V	0.30
Time-critical	0.30% of V	5% of V	0.30
Risk-averse	0.10% of V	2% of V	0.90

These prices are explicit judgments, not learned coefficients or quoted contractual penalties. Eligibility checks scheduled hours, not a sampled arrival; simulated lateness remains a priced outcome. If every schedule misses the deadline, the system returns NoFeasibleRoute and the fastest schedule's shortfall. It does not authorize shipping that fallback.

2. Synthetic catalog and uncertainty

The committed seed-7 catalog contains 30 shipments and 120 candidate routes, cycling through 3, 4, and 5 options. Twelve named ports have illustrative congestion and geopolitical scores. Freight and schedules depend on great-circle distance and transport mode; sea is slow and cheap, air is fast and expensive, and a rail-priced option lies between them. Sea and air use the nearest hub, rail uses the second-nearest, and extra options add detours. Cargo value is log-uniform from $80,000 to $1,500,000. Every tenth shipment deliberately has an impossible deadline.

Each leg draws a delay event at its supplied probability; delay length is exponential with mean hours * (0.15 + 0.60 * congestion). A separate disruption occurs with probability 0.08 * geopolitical risk, adding 96 hours. A disruption damages 30% of remaining cargo with probability 0.25. Congestion and geopolitics are therefore priced through realized time and loss, without an additional risk-label penalty. Shared hops within one shipment reuse the same random draws; different hops are independent. Generation, planning, and evaluation are reproducible from seeds. Full lane formulas are in docs/dataset.md.

3. Explanation and sensitivity

The explanation names the route and hubs, contrasts the freight quote, schedule, and estimated deadline-miss probability with alternatives, and then reconciles the dollar score. Rejected schedules carry a deadline shortfall. A Pareto label compares freight plus expected loss, expected hours, and late probability; it is diagnostic rather than a veto because these three summaries do not capture tail severity.

A concrete example is SHP-005 at seed 7: Dubai to Los Angeles, cargo $86,947, deadline 704 hours. Cost-first selects sea route A. Balanced selects rail-priced route C via Istanbul. C has an 86% higher freight quote than A but a 57% shorter schedule. In the planning sample its late probability is below 0.1%, compared with about 11% for A. C pays $1,059 more in freight and expected loss, but saves $1,174 in holding, $547 in expected lateness, and $1,816 in tail risk: a rounded net advantage of $2,478. Its score is also $5,740 below the fastest air option B. The choice illustrates why the fastest route can be unnecessarily expensive and the cheapest quote can have hidden costs.

Each price is independently scaled by 0.5, 0.67, 0.8, 1.25, 1.5, and 2, using the same simulated arrivals. Duplicate capped tail prices are skipped. The explanation reports switches and choices under all four profiles. This measures preference sensitivity, not uncertainty in the simulator. At seed 42, SHP-015 selects E even though B dominates it on the three summary metrics: E's tail-adjusted score is $546 lower. The explanation reports that small margin and cautions about sampling variability.

4. Evaluation with matching constraints

Planning uses 2,000 scenarios per route. Evaluation uses 1,000 scenarios per route from a distinct random stream under the same model. Each policy sees the same held-out outcomes. Cheapest-feasible minimizes freight among eligible routes; fastest-feasible minimizes scheduled time among those routes. Oracle-feasible chooses the lowest held-out mean landed cost among eligible routes. It is a hindsight reference, not a deployable policy, and its objective excludes the tail premium.

The primary scorecard excludes the three infeasible shipments. Every compared policy obeys the same schedule constraint. Seed 7, 27 feasible shipments:

Policy	Total mean cost	On-time arrivals	Mean regret
Recommended	$179,016	97.34%	$159
Cheapest-feasible	$401,667	85.70%	$8,405
Fastest-feasible	$223,713	99.93%	$1,814
Oracle-feasible	$174,733	96.62%	$0

Total cost sums each shipment's held-out mean landed cost; it is not a freight-only total. On-time rate averages each shipment's simulated fraction of on-time arrivals. Mean regret is average cost above the constrained oracle. Recommended pays 2.45% above the hindsight mean-cost reference while attaining a slightly higher on-time rate. Compared with cheapest-feasible, it reduces total mean cost by 55.4%; compared with fastest-feasible, it reduces cost by 20.0% while accepting 2.59 percentage points fewer on-time arrivals.

Across seeds 7, 11, 21, 42, and 99, recommended on-time rate on feasible shipments ranges from 96.55% to 97.60%, and mean regret from $159 to $394. Recommended Pareto rate is 100% on four seeds and 96.3% on seed 42. Average within-shipment price-probe stability ranges from 95.7% to 97.7%. Results are saved in results/evaluation.json, with sample counts, comparator definitions, and separate feasible-only statistics.

5. Interpreting the full-catalog scorecard

The evaluator also preserves the original unconstrained cheapest and fastest policies to show what happens when schedules are ignored. On seed 7, unconstrained cheapest has total mean landed cost $2,454,826 and 29.74% on-time arrivals. Its mean per-shipment cost ratio against recommended is 8.19, whereas its ratio of total costs is 11.72. These differ because large cargo values and expensive deadline misses have greater weight in totals. Those large ratios also reflect the synthetic 5%-of-value daily penalty for time-critical cargo; they are not evidence of comparable savings in real logistics.

For a forced-shipment comparison across all 30 shipments, recommended and the constrained baselines use the fastest schedule when no schedule is eligible. Each such row is explicitly flagged fallback=true. These are evaluation assumptions, not successful recommendations. With that fallback included, recommended total mean cost is $209,473 and on-time rate is 87.60%. Its actual decision coverage is 27 of 30 shipments, or 90%; the feasible-only on-time rate is 97.34%. Separating these quantities avoids attributing impossible deadlines to poor route selection or presenting abstentions as approved shipments.

6. Design, verification, and limitations

The Python package separates domain types, JSON validation, generation, simulation, pricing and selection, explanations, and evaluation. CLI and Streamlit call the same recommend function. JSON validation catches malformed fields, duplicate shipment IDs, invalid ranges, and inconsistent attributes for an identical hop. The demo shows route comparison, priority overrides, and an infeasible case. Tests include closed-form deterministic costs, risk-sensitive choices, dominance, capped sensitivity probes, CLI exit codes, constraint-matched baselines, fallback exclusion, and interactive shipment changes. The completed local suite has 31 passing tests; the GitHub workflow reruns it on Python 3.12 and 3.13.

The transport topology is deliberately illustrative. Great-circle paths are not navigable sea distances, and the generator can assign rail to intercontinental pairs without a physical rail link. Port risk scores and mode prices are synthetic assumptions, not live assessments, capacity, or carrier quotes. Real deployment requires a mode-valid lane catalog with units, shipment size, schedules, border and transfer rules, availability, and calibrated delay and loss estimates. The parser's identical-hop rule also assumes a single quote context; it cannot represent different carrier or booking-date quotes for the same hop.

Held-out draws test decisions inside the same assumed model. They do not establish robustness to model error or rare events absent from it. Delay and disruption are independent within each hop, and different hops have no regional common shock. Single-price probes omit joint preference shifts. The stated small-margin warning is not a confidence interval. A next step would repeat planning simulations and stress common disruptions, calibrate contractual prices, and report uncertainty bands before relying on near-tie decisions. No learning model is needed without historical outcome labels; a transparent simulator and economic objective are appropriate for this take-home scope.

7. Reproduce and demonstrate

Use Python 3.12 or newer, create a virtual environment, and install from the repository root:

```
pip install -e ".[demo,test,report]"
pytest
python -m routeplan recommend data/shipments.json SHP-005 --seed 7
python -m routeplan evaluate data/shipments.json --seed 7
python scripts/run_evaluation.py
python scripts/render_report.py
streamlit run src/routeplan/app.py
```

The catalog can be regenerated with python -m routeplan generate --seed 7 --shipments 30. docs/demo-script.md gives a recording outline under five minutes: change SHP-005 from cost-first to balanced, show SHP-010's infeasible deadline, then explain the feasible-only scorecard. The README documents setup, JSON input, CLI exit codes, assumptions, and submission files.